Instituto Superior de Engenharia de Coimbra
Instituto Politécnico de Coimbra

Licenciatura Engenharia Informática

Relatótio Trabalho Prático

Sistemas Operativos 25/26

"Gestão de uma Frota de Táxis Autónomos"



Feito por:

- João Bernardo Libório Graça / 2024128702

Data: Dezembro 2025

1 ÍNDICE

2 Introdução III
3 Desenvolvimento e Arquitetura III
3.1 Estrutura Geral do Sistema III
3.2 Mecanismos de Comunicação (IPC) III
3.3 Gestão de Concorrência e Sincronização IV
3.4 Detalhes de Implementação por Módulo V
3.4.1 Controlador V
3.4.2 Veículo V
3.4.3 Cliente V
4 Análise de Funcionalidades V
5 Conclusão VII

2 Introdução

Este relatório descreve o trabalho prático realizado no âmbito da unidade curricular de Sistemas Operativos. O objetivo foi desenhar e implementar uma plataforma de simulação para a gestão de uma frota de táxis autónomos em ambiente Unix/Linux.

O projeto exigiu a aplicação prática de conceitos fundamentais da disciplina, nomeadamente a criação e gestão de processos, a comunicação entre processos (IPC), a manipulação de sinais e a programação concorrente com *threads*.

A solução desenvolvida é composta por três tipos de processos distintos (Controlador, Cliente, Veículo) que interagem de forma coordenada para permitir o agendamento, execução e monitorização de serviços de transporte.

3 Desenvolvimento e Arquitetura

3.1 Estrutura Geral do Sistema

O sistema segue uma arquitetura distribuída centralizada.

- **Controlador:** Atua como o servidor central. Gere a lista de utilizadores, a fila de espera de viagens, a frota ativa e o tempo simulado.
- **Cliente:** Interface para o utilizador final. Permite login, agendamento e interação com o veículo.
- **Veículo:** Processo temporário que simula a execução física do serviço.

O Controlador é responsável por lançar os processos Veículo dinamicamente (usando fork e exec) à medida que as viagens agendadas atingem a hora prevista.

3.2 Mecanismos de Comunicação (IPC)

Para cumprir os requisitos do enunciado, foram utilizados diferentes mecanismos de comunicação consoante a necessidade:

1. **Named Pipes (FIFOs):** Utilizados para a comunicação assíncrona entre processos independentes.
    - **Canal Público (/tmp/taxi_fifo):** Usado por todos os Clientes para enviar pedidos ao Controlador.
    - **Canais Privados (/tmp/cli_<pid>_fifo):** Criados por cada Cliente para receber respostas do Controlador e notificações diretas dos Veículos.
2. **Pipes Anónimos e Redirecionamento:** Utilizados para a "telemetria". O Controlador cria um pipe anónimo antes de lançar um Veículo e redireciona o stdout do filho para a escrita do pipe. Isto permite ao Controlador ler em tempo real os printf feitos pelo Veículo.
3. **Sinais (Signals):**
    - SIGINT: Para encerramento limpo (apanhar Ctrl+C) e remoção dos FIFOs.
    - SIGCHLD: Para o Controlador detetar a terminação de Veículos e evitar processos *zombie*.
    - SIGUSR1: Enviado pelo Controlador ao Veículo para forçar o cancelamento imediato de uma viagem.

3.3 Gestão de Concorrência e Sincronização

Uma das principais decisões de arquitetura foi a utilização de **Threads POSIX (pthreads)** no Controlador e no Cliente para lidar com múltiplas fontes de entrada (Input).

- **No Controlador:** Em vez de utilizar *polling* com sleep ou o mecanismo select (proibido pelo enunciado), optou-se por uma arquitetura multi-threaded:
    - *Thread Principal:* Gere o relógio, verifica agendamentos e lança veículos.
    - *Thread Clientes:* Bloqueia na leitura do FIFO principal.
    - *Thread Admin:* Bloqueia na leitura do teclado (comandos de gestão).

- **Sincronização:** Como as threads partilham memória global (listas de clientes e viagens), foi implementado um **Mutex** (pthread_mutex_t) para garantir a exclusão mútua e evitar *race conditions* durante a manipulação das estruturas de dados.

3.4 Detalhes de Implementação por Módulo

3.4.1 Controlador

O Controlador implementa uma lógica de agendamento inteligente. Se a frota estiver cheia (NVEICULOS), o pedido não é rejeitado, mas sim colocado em espera. O sistema recalcula a próxima vaga disponível com base na estimativa de fim das viagens em curso.

Para a leitura da telemetria dos veículos, manteve-se o uso de I/O não-bloqueante (O_NONBLOCK) nos pipes anónimos, permitindo ao relógio avançar mesmo que os veículos não enviem dados num determinado segundo.

3.4.2 Veículo

O Veículo é um processo independente. Um desafio técnico superado foi o buffering do stdout quando ligado a pipes. Foi utilizada a função setbuf(stdout, NULL) para garantir que as mensagens de telemetria são enviadas imediatamente para o Controlador, evitando situações de deadlock.

3.4.3 Cliente

O Cliente utiliza duas threads para separar a leitura do teclado da receção de mensagens, permitindo uma interface reativa onde notificações (ex: "Veículo Chegou") podem aparecer enquanto o utilizador escreve um comando.

4 Análise de Funcionalidades

A tabela seguinte resume o estado de implementação dos requisitos funcionais propostos no enunciado:

Módulo	Funcionalidade	Estado	Observações

Geral	Compilação Automática	✅ Cumprido	Implementado via Makefile.
Controlador	Unicidade	✅ Cumprido	Deteta se o FIFO já existe.
	Gestão de Frota	✅ Cumprido	Respeita NVEICULOS e gere fila de espera.
	Gestão de Tempo	✅ Cumprido	Relógio simulado.
	Lançamento Veículos	✅ Cumprido	fork + exec + argumentos.
	Telemetria	✅ Cumprido	Redirecionamento de stdout.
	Comandos Admin	✅ Cumprido	utiliz, listar, frota, km, cancelar.
Cliente	Login / Identificação	✅ Cumprido	
	agendar, consultar	✅ Cumprido	

	cancelar <id>	✅ Cumprido	Cancela viagens pendentes por ID.
	entrar	✅ Cumprido	Simplificação aceite pelo docente.
Veículo	Simulação	✅ Cumprido	
	Cancelamento	✅ Cumprido	Reage a SIGUSR1 terminando.

5 Conclusão

O trabalho prático foi concluído com sucesso, resultando num sistema funcional que cumpre todos os requisitos obrigatórios e opcionais.

A adoção de uma arquitetura baseada em Threads para a gestão de entradas (I/O) revelou-se uma escolha eficiente, simplificando o código de leitura e melhorando a reatividade do sistema face a uma abordagem de *polling*. A combinação de mecanismos clássicos de Unix (Processos e Sinais) com mecanismos de concorrência (Threads e Mutexes) permitiu consolidar de forma prática a matéria lecionada.

O sistema final é robusto, capaz de recuperar de erros de comunicação (clientes que terminam abruptamente) e de gerir recursos limitados (frota) de forma autónoma.